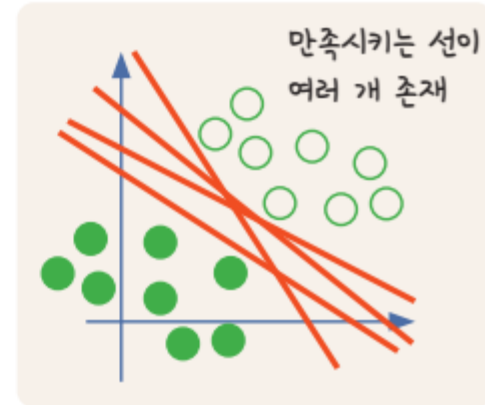
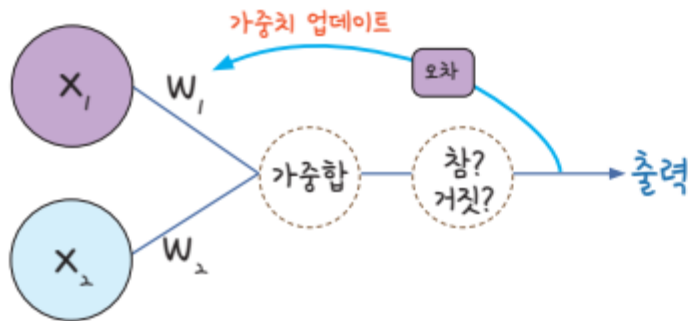


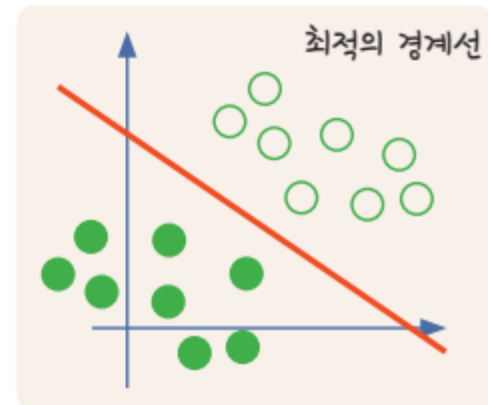
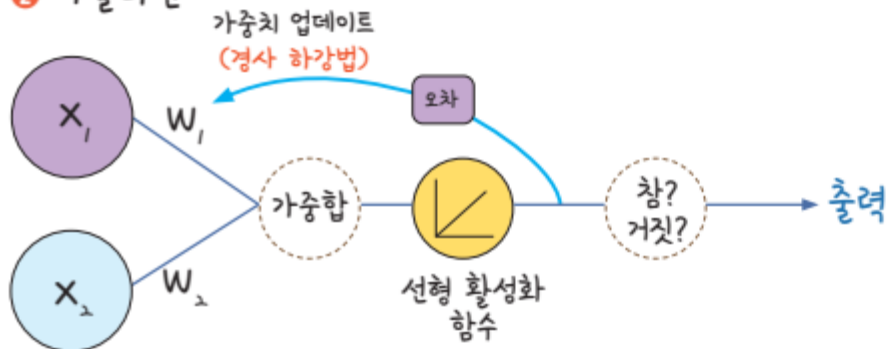
1 인공지능의 시작을 알린 퍼셉트론

▼ 그림 7-2 | 퍼셉트론, 아달라인 그리고 로지스틱 회귀 모델

1 퍼셉트론



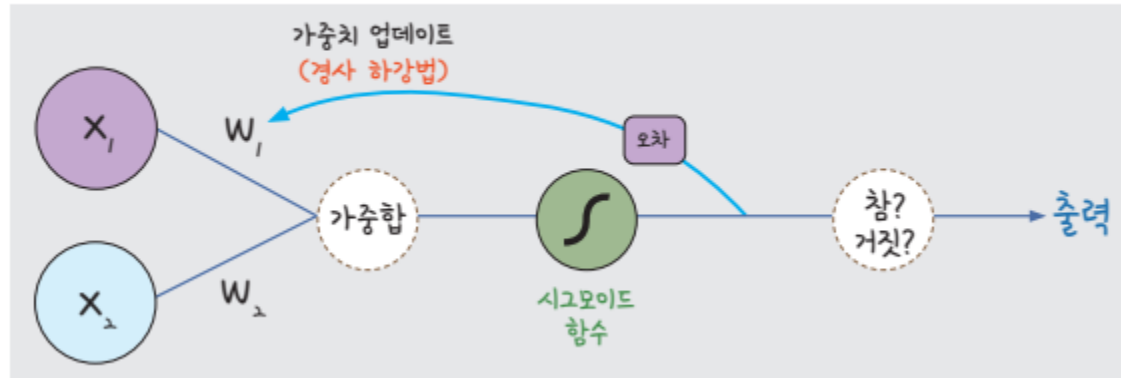
2 아달라인



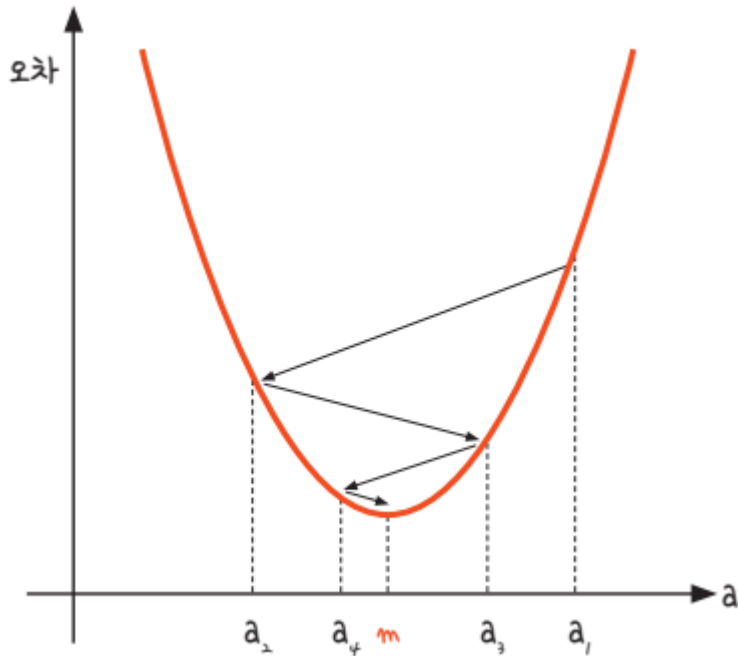
1 인공지능의 시작을 알린 퍼셉트론

▼ 그림 7-2 | 퍼셉트론, 아달라인 그리고 로지스틱 회귀 모델(계속)

(비교) 로지스틱 회귀 모델



경사하강법



이렇게 구한 a 와 b 의 편미분 결과는 경사하강법에서 모델의 가중치를 업데이트할 때 다음과 같이 사용됩니다.

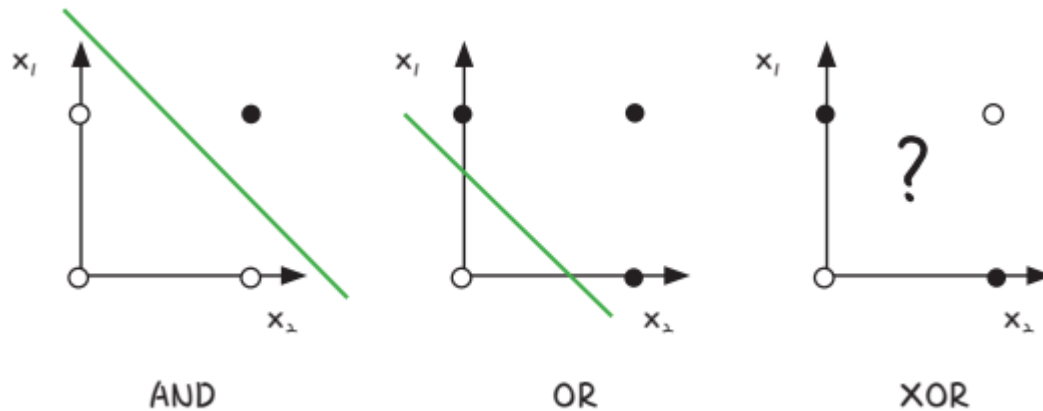
- $a \leftarrow a - \eta \frac{\partial E}{\partial a}$
- $b \leftarrow b - \eta \frac{\partial E}{\partial b}$ (단, η 는 학습률)

3 XOR 문제

- XOR 문제

- 그림 7-5를 각각 그래프로 좌표 평면에 나타내 보겠음
- 결괏값이 0이면 흰색 점으로, 1이면 검은색 점으로 나타낸 후 조금 전처럼 직선을 그어 위 조건을 만족할 수 있는지 보겠음

▼ 그림 7-6 | AND, OR, XOR 진리표대로 좌표 평면에 표현한 후 선을 그어 색이 같은 점끼리 나누기(XOR는 불가능)

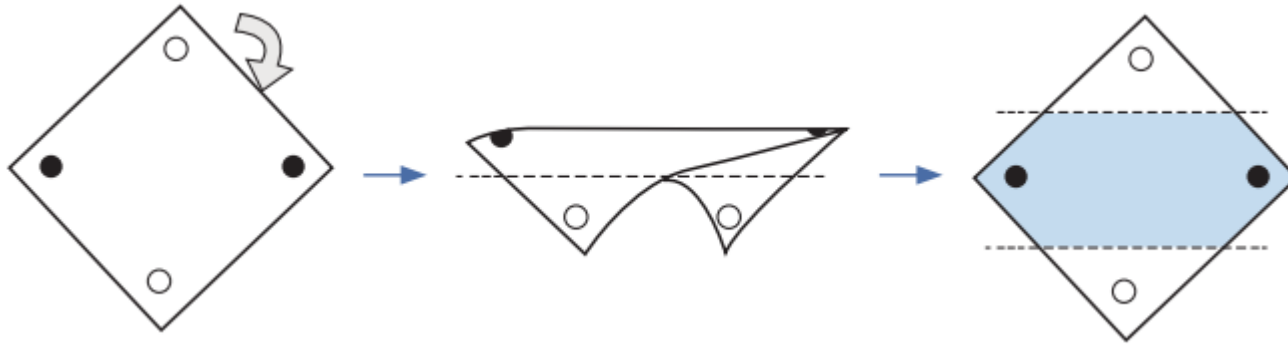


1 다층 퍼셉트론의 등장

● 다층 퍼셉트론의 등장

- 인공지능 학자들은 인공 신경망을 개발하기 위해 반드시 XOR 문제를 극복해야만 했음
- 이 문제를 해결하는 데도 고정 관념을 깨는 기발한 아이디어가 필요했음

▼ 그림 8-3 | XOR 문제의 해결은 평면을 휘어 주는 것!



- 즉, 종이를 휘어서 선 두 개를 동시에 긋는 방법
- 이것을 XOR 문제에 적용하면 ‘퍼셉트론 두 개를 한 번에 계산’하면 된다는 결론에 이룸
- 이를 위해 퍼셉트론 두 개를 각각 처리하는 은닉층(hidden layer)을 만들

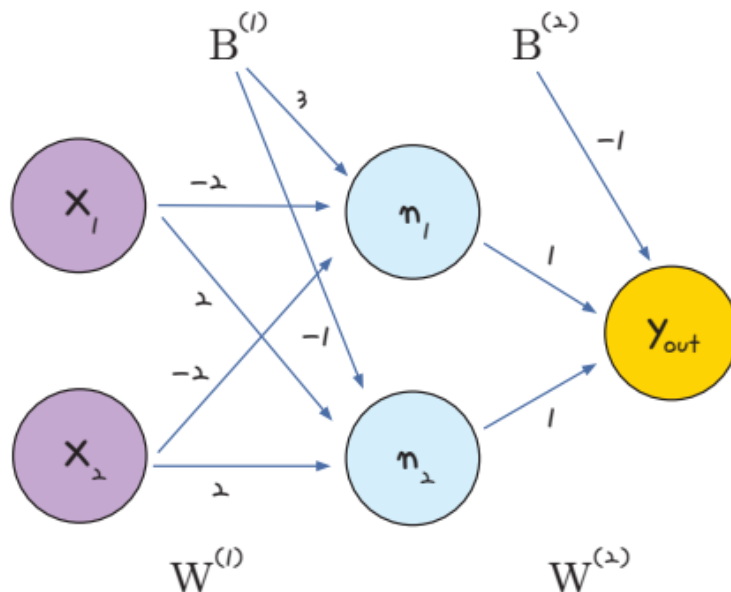
3 XOR 문제의 해결

- XOR 문제의 해결

- 다음과 같이 각 변수 값을 정하고 이를 이용해 XOR 문제를 해결하는 과정을 알아보자

$$\begin{aligned} W^{(1)} &= \begin{bmatrix} -2 & 2 \\ -2 & 2 \end{bmatrix} & B^{(1)} &= \begin{bmatrix} 3 \\ -1 \end{bmatrix} \\ W^{(2)} &= \begin{bmatrix} 1 \\ 1 \end{bmatrix} & B^{(2)} &= [-1] \end{aligned}$$

▼ 그림 8-6 | 다층 퍼셉트론의 내부에 변수 채우기



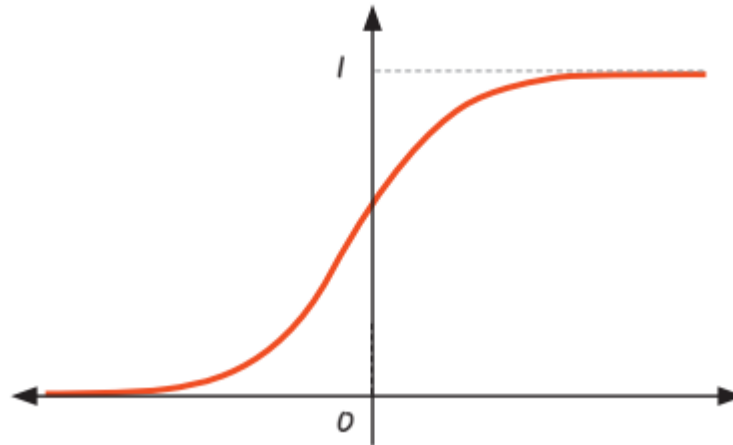
3 XOR 문제의 해결

● XOR 문제의 해결

- 이제 x_1 값과 x_2 값을 각각 입력해 우리가 원하는 y 값이 나오는지 점검해 보자

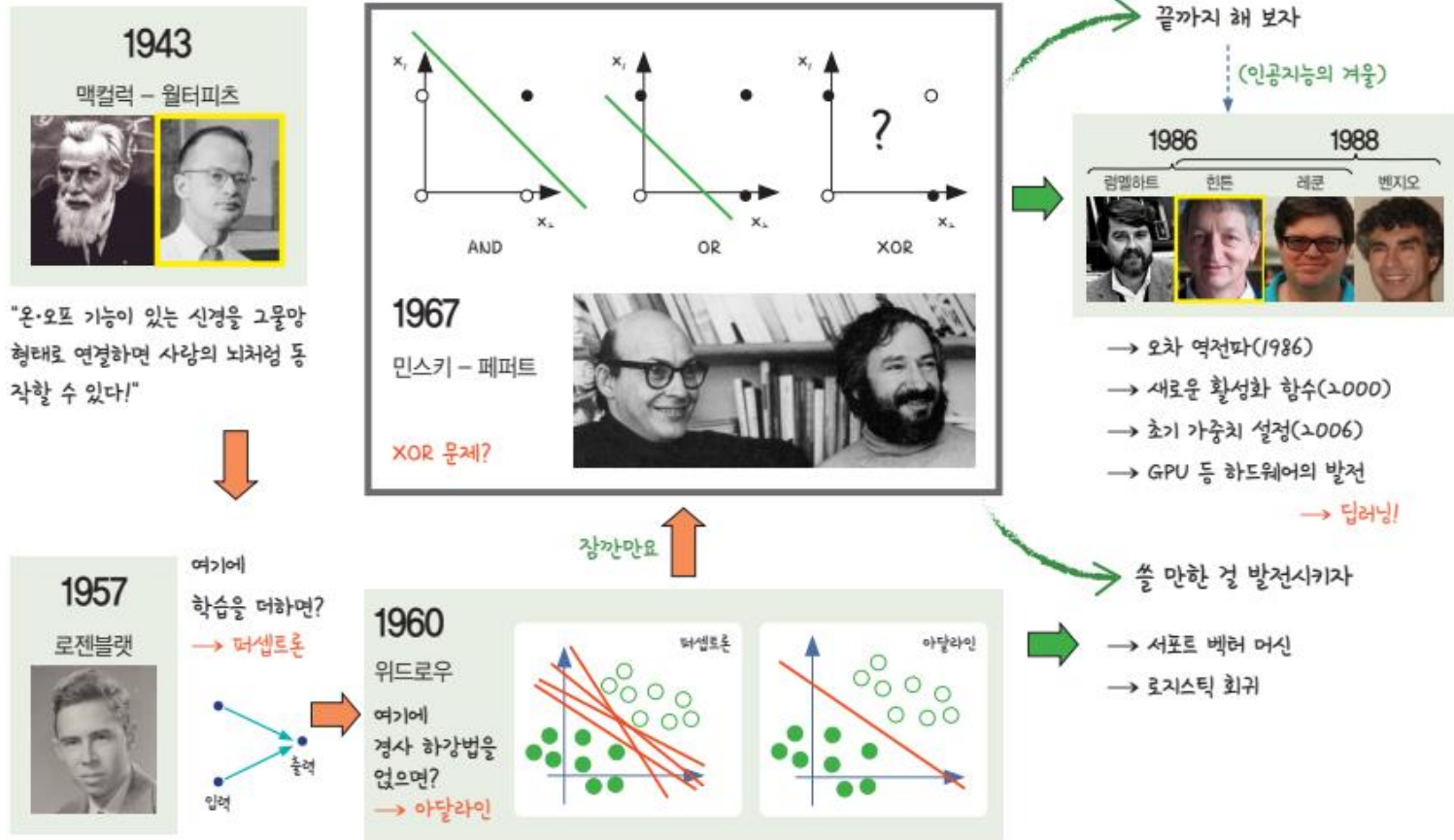
▼ 표 8-1 | XOR 다층 문제 해결

x_1	x_2	n_1	n_2	y_{out}	우리가 원하는 값
0	0	$\sigma(0 * (-2) + 0 * (-2) + 3) \approx 1$	$\sigma(0 * 2 + 0 * 2 - 1) \approx 0$	$\sigma(1 * 1 + 0 * 1 - 1) \approx 0$	0
0	1	$\sigma(0 * (-2) + 1 * (-2) + 3) \approx 1$	$\sigma(0 * 2 + 1 * 2 - 1) \approx 1$	$\sigma(1 * 1 + 1 * 1 - 1) \approx 1$	1
1	0	$\sigma(1 * (-2) + 0 * (-2) + 3) \approx 1$	$\sigma(1 * 2 + 0 * 2 - 1) \approx 1$	$\sigma(1 * 1 + 1 * 1 - 1) \approx 1$	1
1	1	$\sigma(1 * (-2) + 1 * (-2) + 3) \approx 0$	$\sigma(1 * 2 + 1 * 2 - 1) \approx 1$	$\sigma(0 * 1 + 1 * 1 - 1) \approx 0$	0



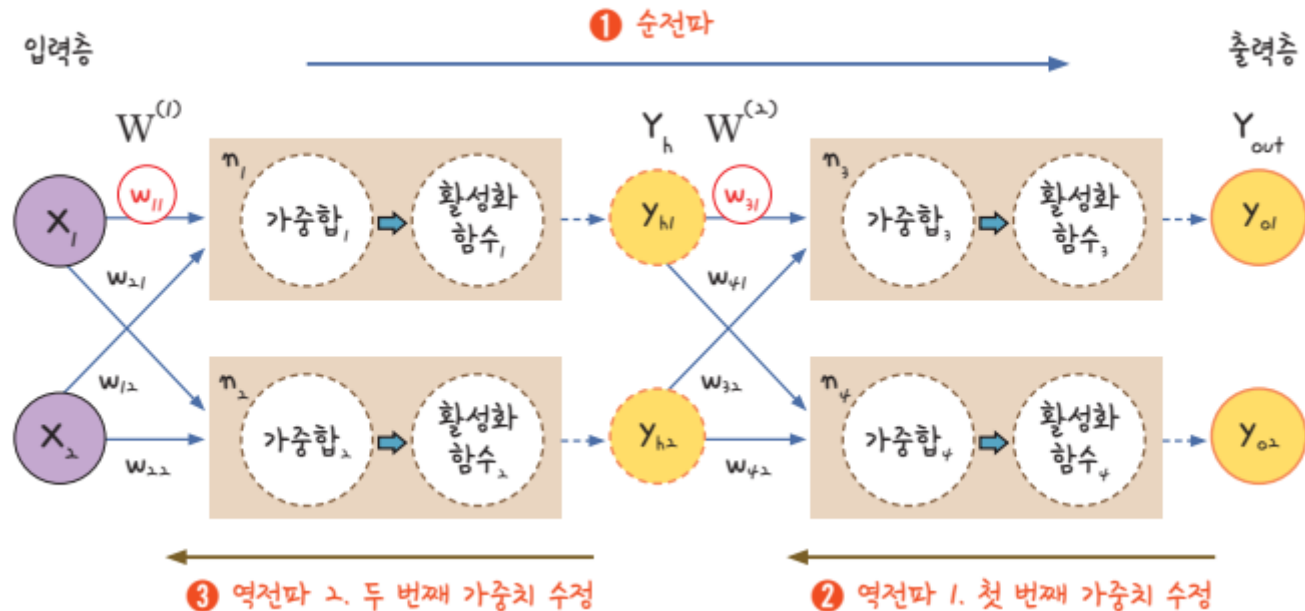
4 코딩으로 XOR 문제 해결하기

▼ 그림 8-7 | 한눈에 보는 인공지능의 역사: 퍼셉트론에서 딥러닝까지



1 딥러닝의 태동, 오차 역전파

▼ 그림 9-3 | 오차 역전파의 개념

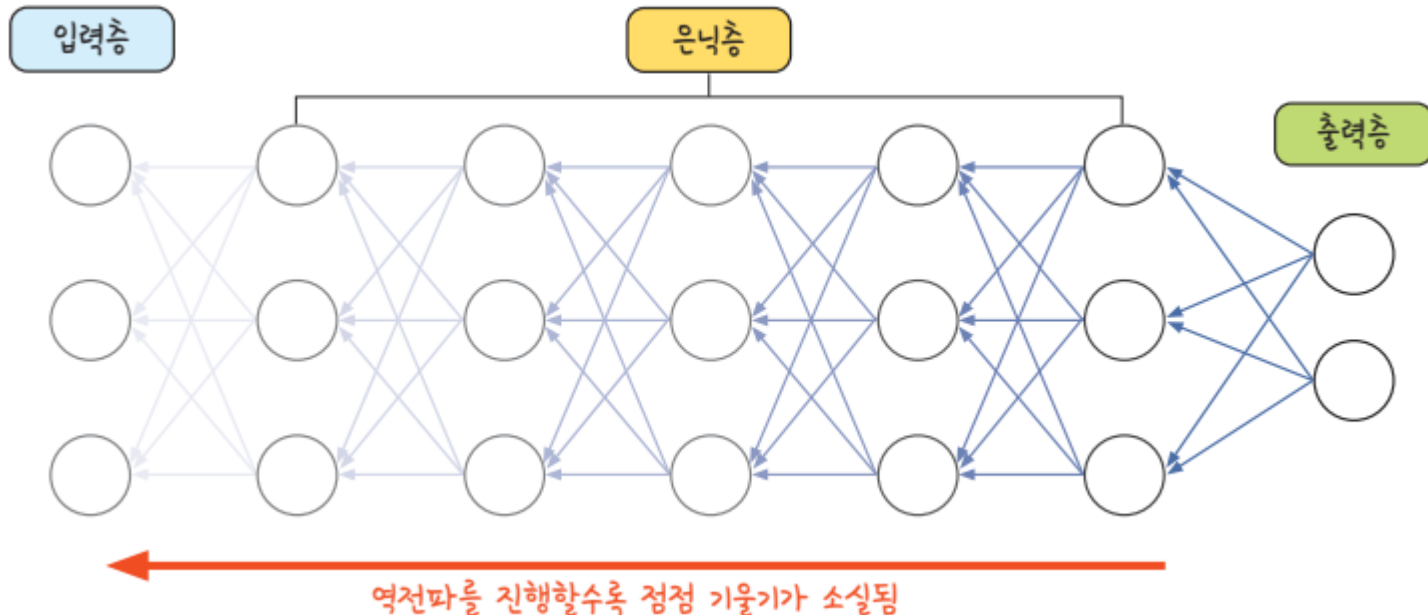


2 기울기 소실 문제와 활성화 함수

● 기울기 소실 문제와 활성화 함수

- 앞서 델타식을 이용해 깊은 신경망의 계산이 가능해졌음을 이야기했음
- 이제 수많은 층을 연결해 학습하면 여러 난제를 해결하는 인공지능이 완성될 것 같아 보이지만 아직 한 가지 문제가 더 남아 있음

▼ 그림 9-4 | 기울기 소실 문제 발생

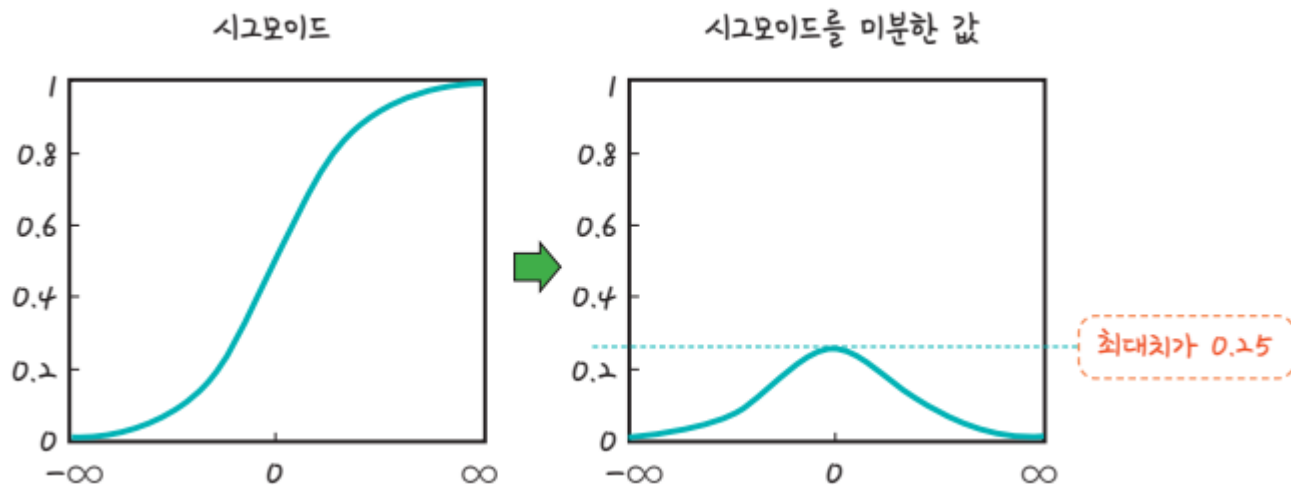


2 기울기 소실 문제와 활성화 함수

● 기울기 소실 문제와 활성화 함수

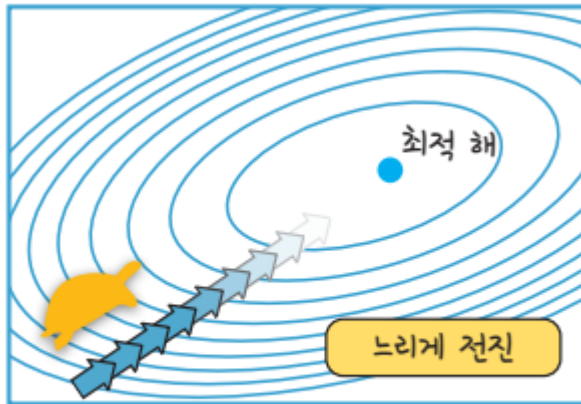
- 그림 9-4와 같이 깊은 층을 만들어 보니 출력층에서 시작된 가중치 업데이트가 처음 층까지 전달되지 않는 현상이 생기는 문제가 발견
- 이는 활성화 함수로 사용된 시그모이드 함수의 특성 때문임
- 그림 9-5와 같이 시그모이드 함수를 미분하면 최대치는 0.25
- 1보다 작으므로 계속 곱하다 보면 0에 가까워짐
- 여러 층을 거칠수록 기울기가 사라져 가중치를 수정하기 어려워지는 것

▼ 그림 9-5 | 시그모이드의 미분

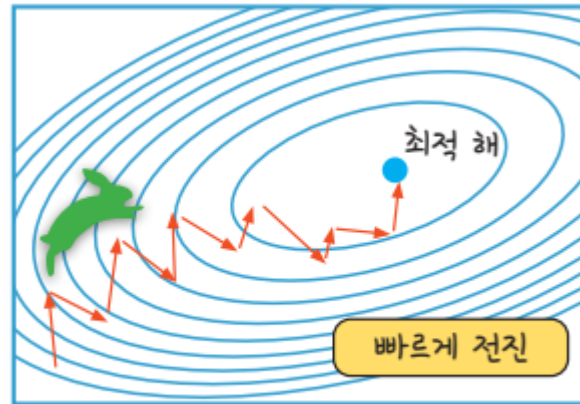


3 속도와 정확도 문제를 해결하는 고급 경사 하강법

▼ 그림 9-7 | 경사 하강법과 확률적 경사 하강법의 비교



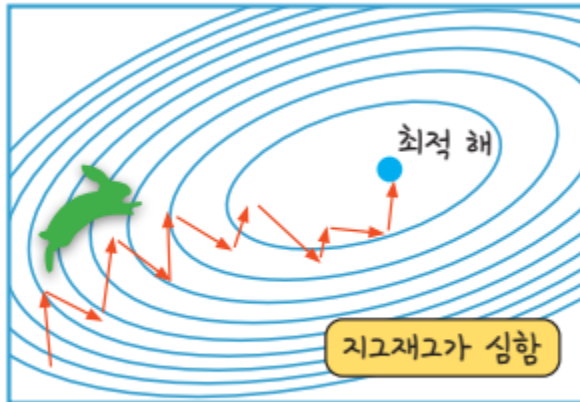
경사 하강법



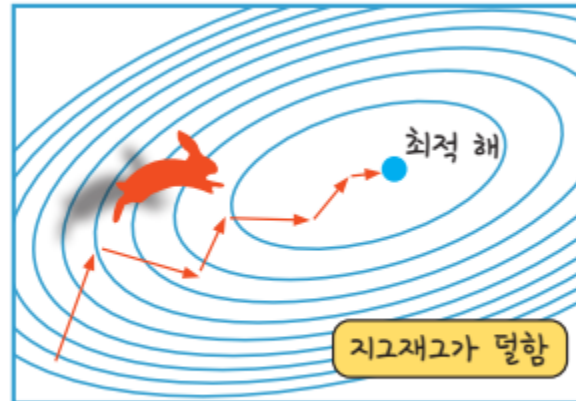
확률적 경사 하강법

3 속도와 정확도 문제를 해결하는 고급 경사 하강법

▼ 그림 9-8 | 모멘텀을 적용했을 때



확률적 경사 하강법



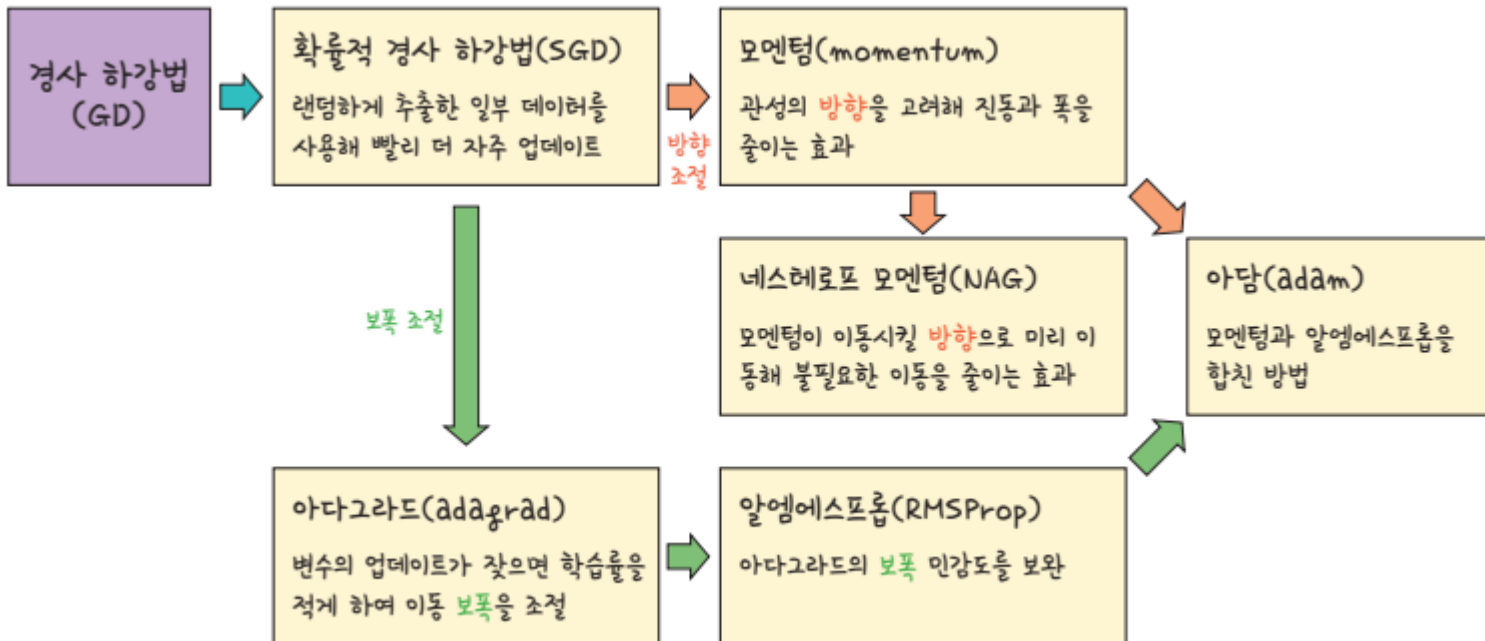
모멘텀을 적용한 확률적 경사 하강법

3 속도와 정확도 문제를 해결하는 고급 경사 하강법

● 속도와 정확도 문제를 해결하는 고급 경사 하강법

- 이 밖에도 딥러닝의 학습을 더 빠르고 정확하게 만들기 위한 노력이 계속됨
- 지금은 정확도와 속도를 모두 향상시킨 아담(adam)이라는 고급 경사 하강법이 가장 많이 쓰이고 있음

▼ 그림 9-9 | 딥러닝에 사용되는 고급 경사 하강법의 변천



오차역전파와 체인룰

2. 연쇄법칙(chain rule)

chain rule은 합성 함수 미분법의 성질을 말한다. 합성 함수를 미분할 때 꼬리에 꼬리를 물며 겉함수에서 속함수를 곱하는 과정을 거치는데 이를 chain이라 표현한 것이다. 예를 들어 다음과 같은 두 합성 함수에 대한 미분 과정을 생각해보자.

겉미분*속미분

$$f = (x + y)^2 \rightarrow f' = 2(x + y) \cdot 1$$

$$f = \{(x + y)^3 + z\}^2 \rightarrow f' = 2\{(x + y)^3 + z\} \cdot 3(x + y) \cdot 1$$

합성 함수 f를 x에 대해 미분하는 과정

오차역전파와 체인룰

이 합성 함수의 미분 과정을 수식으로 표현하면 다음과 같다.

$$\begin{array}{ll} z = g(y), y = f(x) & \text{미분} \\ & \rightarrow (g \circ f)'(x) = g'(y)f'(x) \\ z = (g \circ f)(x) = g(f(x)) & \downarrow \text{Chain rule} \end{array}$$

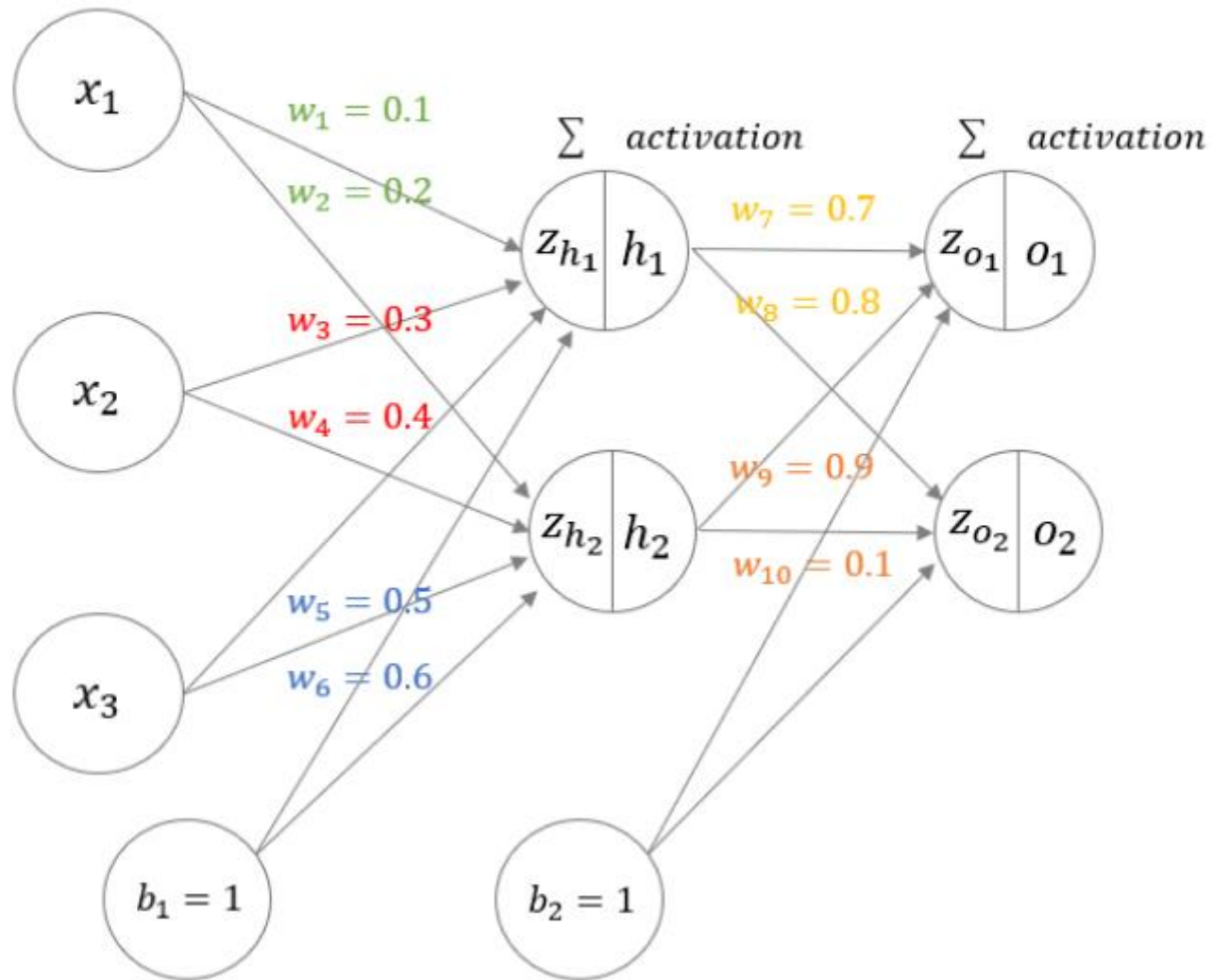
$$\frac{dz}{dx} = \frac{dz}{dy} \cdot \frac{dy}{dx}$$

chain rule

z함수를 x에 대해 미분한 결과는 z함수를 y에 대해 미분한 결과와 y함수를 x에 대해 미분한 결과를 곱하여 구할 수 있다는 것이 핵심이다.

위의 chain rule을 신경망에서 기울기를 계산할 때 적용해보도록 하자.

오차역전파와 체인룰



오차역전파와 체인룰

- $E = \frac{1}{2}[(o_1 - t_1)^2 + (o_2 - t_2)^2]$
- $z_{h_1} = w_1x_1 + w_3x_2 + w_5x_3 + b_1$
- $z_{h_2} = w_3x_1 + w_4x_2 + w_6x_3 + b_1$
- $h_1 = a(z_{h_1})$
- $h_2 = a(z_{h_2})$
- $z_{o_1} = w_7h_1 + w_9h_2 + b_2$
- $z_{o_2} = w_8h_1 + w_{10}h_2 + b_2$
- $o_1 = a(z_{o_1})$
- $o_2 = a(z_{o_2})$

예를 들어, chain rule성질을 이용하여 손실 함수 E를 w_7 에 대해 미분을 하면 아래의 식과 같다.

$$\frac{\partial E}{\partial w_7} = \frac{\partial E}{\partial o_1} \times \frac{\partial o_1}{\partial z_{o_1}} \times \frac{\partial z_{o_1}}{\partial w_7}$$

chain rule을 이용하여 계산한 w_7 에 대한 Error함수 미분

손실 함수의 값을 줄이기 위해 weight를 갱신하는 과정을 간략하게 정리하면,

먼저 weight들의 초기값들로 forward pass를 한다. 그 후 backpropagation을 통해 손실 함수를 각 weight에 대해 미분한 값을 구하여 그 미분값과 learning rate와 minus부호를 곱하여 새로운 weight로 갱신한다.

$$W = W - learning\ rate * \frac{\partial L}{\partial W}$$

Weight 갱신